

# Advanced Programming Techniques

**fbeit**

FACHBEREICH ELEKTROTECHNIK  
UND INFORMATIONSTECHNIK

**Prof. Dr. Michael Lipp**

Question <https://poll.mnl.de> (Code: )

- **Operator overloading ...**
  1. Can only be used for unary operators
  2. Can only be used for ternary operators
  3. Comes in two flavors
  4. Makes coding easier for the author of a class

Question <https://poll.mnl.de> (Code: )

- Operator overloading with method (choose best match) ...
  1. May be used to overload the “output operator”
  2. May be used to overload post-increment operator
  3. May be used to overload assignment operator

**fbeit**

FACHBEREICH ELEKTROTECHNIK  
UND INFORMATIONSTECHNIK

# Where we are

# Where we are

- You've learned ...
  - what objects and classes are,
  - how they are used to model real (or virtual) world “things”
    - (in order to handle them in a program),
  - how to define classes with their attributes and methods in the programming language C++,
  - how to make sure that objects
    - have defined state when newly created and
    - release resources when they are deleted,
  - how memory is allocated for attributes (both statically sized and dynamically sized),
  - how object can be used more comfortably by supporting operators.



Doesn't fully work yet

# Rules of X

## Memory Management II

**fbeit**

FACHBEREICH ELEKTROTECHNIK  
UND INFORMATIONSTECHNIK

# Problems copying/assigning objects

- Try this:

```
SimpleVector a{10};  
a.set(5, 2.1);  
SimpleVector b = a;  
a.print();  
b.print();  
a.set(5, 4.2);    // Change a  
b.print();        // Print b
```

- Unexpected output!

```
Vector a:  
{ 0, 0, 0, 0, 0, 2.1, 0, 0, 0, 0 }  
Vector b:  
{ 0, 0, 0, 0, 0, 2.1, 0, 0, 0, 0 }  
Vector b after changing a:  
{ 0, 0, 0, 0, 0, 4.2, 0, 0, 0, 0 }  
free(): double free detected in tcache 2
```

Changing a shouldn't  
change b!

What's this?

# Copying objects

- **Copies of objects are created**
  - explicit (1): `SimpleVector b{a};`
  - explicit (2): `SimpleVector b = a;`
    - Looks like an assignment but is really an initialization, i.e. the compiler generates a new object using the values from `b` (what good would it be to first invoke the default constructor and then overwrite everything?)
  - by passing an object as argument (see `Fraction::add`)
  - when returning an object from a method (temporary value) (see `Fraction::add`)
- **Copies are created by invoking the “Copy constructor”**



# Copy constructor

- **Special constructor**

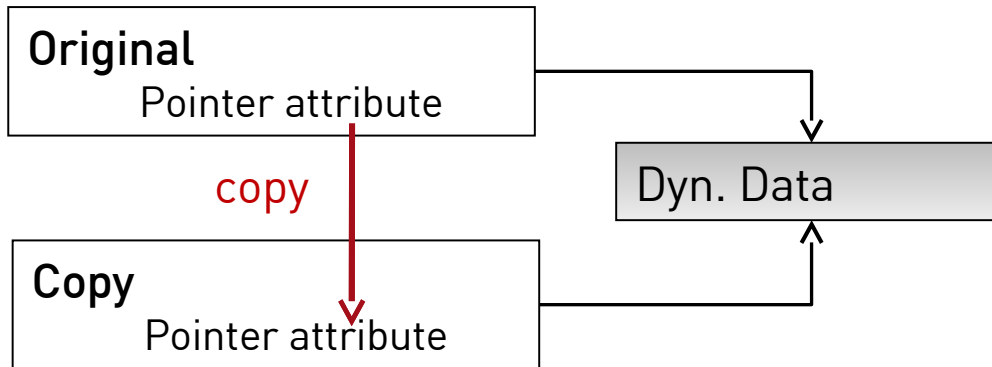
- Takes another instance as parameter
- Copies the state from the “original” object to the newly created object
- For SimpleVector:

```
SimpleVector(const SimpleVector& orig);
```

- **If not provided by class definition**

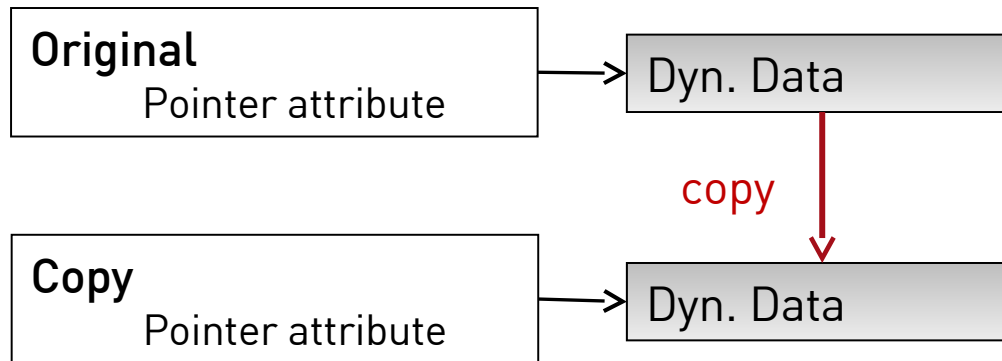
- Automatically created by compiler
- Behavior: memberwise copy of data members (values only, “shallow copy”)

# Shallow copy vs. deep copy



- **Shallow copy**

- Changes of the dynamic data made by either the original or the copied object are visible to both



- **Deep copy**

- Changes of the dynamic data made by either the original or the copied object are effect only the object that made the change

# Copy Constructor for SimpleVector

```
SimpleVector::SimpleVector(const SimpleVector &orig):
```

```
    size{orig.size},
```

Copy initialization

```
    elements{new double[orig.size]} {
```

Initialization with  
new value

```
    for (int i = 0; i < size; i++) {
```

```
        elements[i] = orig.elements[i];
```

```
    }
```

```
}
```

Copy data from original  
heap memory to new  
heap memory (copy)

# Assignment

- Now what about this:

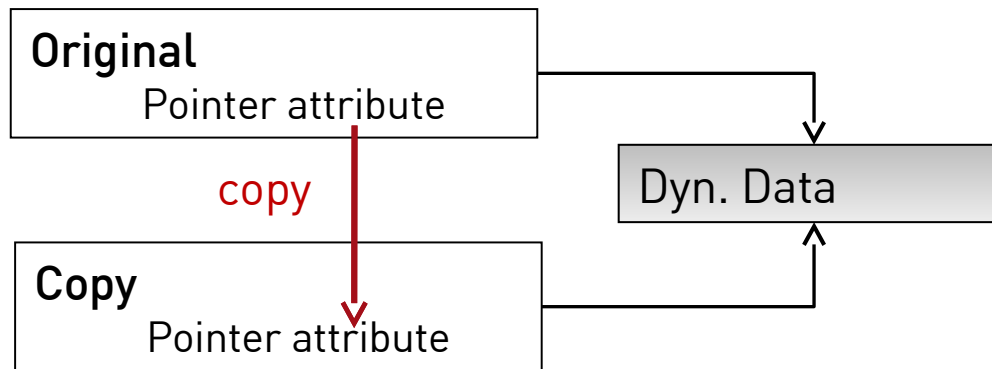
```
SimpleVector c{10};  
c = b;  
cout << "Vector c after assignment:" << endl;  
c.print();  
b.set(5, 123); // Change b  
cout << "Vector c after changing b:" << endl;  
c.print();     // Print c
```

- Unexpected output:

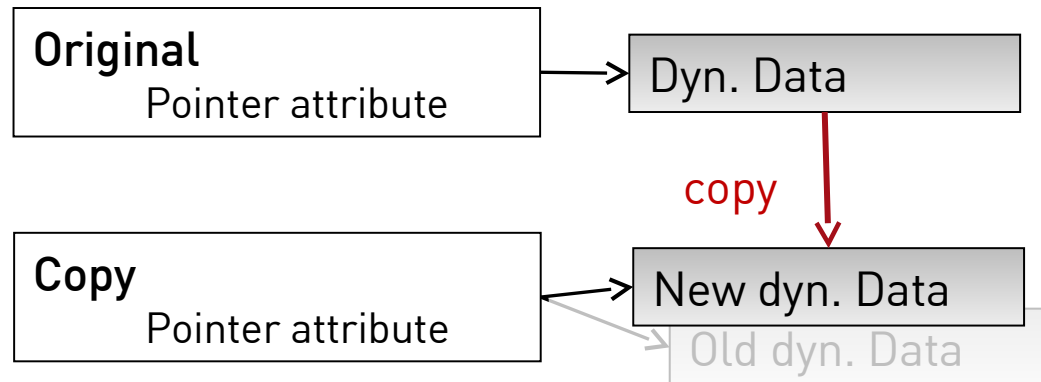
```
Vector c after assignment:  
{ 0, 0, 0, 0, 0, 2.1, 0, 0, 0, 0 }  
Vector c after changing b:  
{ 0, 0, 0, 0, 0, 123, 0, 0, 0, 0 }  
free(): double free detected in tcache 2
```

Changing b shouldn't  
change c!

# Copy Assignment



- “Shallow assignment”
  - Changes of the dynamic data made by either the original or the copied object are visible to both



- Deep copy on assignment
  - Changes of the dynamic data made by either the original or the copied object are effect only the object that made the change. **Old memory must be deleted!**

# Copy Assignment for SimpleVector

```
SimpleVector& SimpleVector::operator=(  
    const SimpleVector &orig) {  
    delete [] elements;  
    size = orig.size;  
    elements = new double[size];  
    for (int i = 0; i < size; i++) {  
        elements[i] = orig.elements[i];  
    }  
    return *this;  
}
```

Free currently used  
heap memory

Set data members as  
in copy constructor

Copy data from original  
heap memory to new  
heap memory (copy)

Return target for further usage in expressions

# Rule of three (C++98)

- **If a class needs**
  - a destructor or a
  - copy constructor or an
  - overloaded copy assignment operator
- **then it also needs the other two.**

# Temporary Objects

- Look at this code:

```
SimpleVector sumIntoFirst(SimpleVector v) {  
    for (int i = 1; i < 10; i++) {  
        v.set(0, v.get(0) + v.get(i));  
    }  
    return v;  
}
```

```
SimpleVector d;  
d = sumIntoFirst(c);
```

- How often is **SimpleVector** copied?

Check in Eclipse  
debugger



# Temporary Objects

- Look at this code:

```
SimpleVector sumIntoFirst(SimpleVector v) {  
    for (int i = 1; i < 10; i++) {  
        v.set(0, v.get(0) + v.get(i));  
    }  
    return v;  
}
```

```
SimpleVector d;  
d = sumIntoFirst(c);
```

- How often is **SimpleVector** copied?
  - When passing the argument (copy into parameter)

# Temporary Objects

- Look at this code:

```
SimpleVector sumIntoFirst(SimpleVector v) {  
    for (int i = 1; i < 10; i++) {  
        v.set(0, v.get(0) + v.get(i));  
    }  
    return v;  
}
```

```
SimpleVector d;  
d = sumIntoFirst(c);
```

- How often is **SimpleVector** copied?
  - When passing the argument (copy into parameter)
  - When returning the value (temporary object)

# Temporary Objects

- Look at this code:

```
SimpleVector sumIntoFirst(SimpleVector v) {  
    for (int i = 1; i < 10; i++) {  
        v.set(0, v.get(0) + v.get(i));  
    }  
    return v;  
}
```

```
SimpleVector d;  
d = sumIntoFirst(c);
```

- How often is **SimpleVector** copied?
  - When passing the argument (copy into parameter)
  - When returning the value (temporary object)
  - When copy assigning from temporary to result container

# Temporary Objects

- Look at this code:

```
SimpleVector sumIntoFirst(SimpleVector v) {  
    for (int i = 1; i < 10; i++) {  
        v.set(0, v.get(0) + v.get(i));  
    }  
    return v;  
}
```

```
SimpleVector d;  
d = sumIntoFirst(c);
```

- How often is **SimpleVector** copied?

- Copy constructor twice
- Copy assignment once



Costly! Imagine vector with 100000 elements

# Copying details

- **The copies**

- 1) Copy of c passed into function: required, we change v
- 2) Copy of calculated result (v) into temporary object: only needed until returned value has been processed
- 3) Copy into d: source no longer needed after copy

- **Source of copy operations 2) and 3) no longer needed after copying**

- **Optimizing opportunity:**

- Move resources to target instead of copying
- Leave source object in “destructable” state

# Move constructor and move assignment

- Move constructor

```
SimpleVector::SimpleVector(SimpleVector &&source) {  
    size = orig.size;  
    orig.size = 0;  
    elements = orig.elements;  
    orig.elements = nullptr;  
}
```

Steal resources from source  
and then ourselves

- Move assignment

```
SimpleVector& SimpleVector::operator =(SimpleVector &&source) {  
    std::swap(size, source.size);  
    std::swap(elements, source.elements);  
    return *this;  
}
```

Swap resources. What we used  
in target will be freed when  
source is eventually deleted

# Move constructor and move assignment

- Note that the move constructor and move assignment violate that usual rule that creating a new object from some source object or assigning from some source object does not modify the source object
- However, this does not matter because the source objects are temporary objects and no longer used after the operation

# Rule of five (C++11)

- If a class needs
  - a destructor or a
  - copy constructor or an
  - overloaded copy assignment operator
- then it also needs the other two
- and should define
  - a move constructor and a
  - move assignment operator



# Rule of zero

- **Simplified version of rule of zero:**
  - Avoid all problems by using in own class only attributes of types that already have “proper” behavior
- **In this case the default implementations for the five methods provided by the compiler are okay**
  - We don’t have to provide own implementations
- **Predefined types with “proper” behavior can be found for almost all tasks in the standard library (next lecture)**



May not be used in first lab exercise